Lab Handout 1-E
Field-Programmable Gate Array (FPGA) Toolchain Validation: Nexys A7 Loopback Designs

Capstone — Module 1: Radio Frequency (RF) Hardware Foundations

1 Objective

Validate the AMD Vivado toolchain and the Digilent Nexys A7-100T board by synthesizing, implementing, and programming two loopback designs: (1) a trivial switch-to-LED loopback that proves the Vivado flow end-to-end, and (2) a UART echo loopback that proves the USB-UART path that Module 2 will rely on. Both must pass before moving on to Register-Transfer Level (RTL) development.

2 Prerequisites

- No prior lab handouts required; runs in parallel to the STM32 bring-up track.
- Basic VHSIC Hardware Description Language (VHDL) syntax familiarity (see Module 1 Prerequisites for references).

3 Materials

- Digilent Nexys A7-100T development board (XC7A100T-1CSG324C).
- USB Micro-B cable.
- Host PC with $\geq$50 GB free disk space for Vivado.
- Serial terminal program (PuTTY, Tera Term, or similar).

4 Procedure

4.1 Step 1 — Install Vivado (first-time only)

1. Download Vivado ML Standard (the free edition that covers Artix-7) from xilinx.com/support/download.html. Create a free AMD account if you do not already have one.
2. Run the installer. When prompted for device families, select **Artix-7** and **7 Series** at minimum; deselecting other families saves tens of GB.
3. On Windows, let the installer install the cable drivers when prompted at the end.
4. Launch Vivado once to verify the license. The free Standard edition does not require a license file for Artix-7 devices.

[Figure placeholder]

`LH1E_vivado_installer.png`

*Vivado installer device-family selection screen with **Artix-7** and **7 Series** checked.*

Figure 1: Vivado installer device-family selection screen with **Artix-7** and **7 Series** checked.

4.2 Step 2 — Obtain the master constraint file

1. Download `Nexys-A7-100T-Master.xdc` from Digilent's Nexys A7 reference page (search: "Digilent Nexys A7 master XDC").
2. Save it locally. You will add it to the project and uncomment only the pins you need.

4.3 Step 3 — Create the Vivado project

1. Vivado → *File* → *Project* → *New*. Name the project `nexys_loopback`.
2. Project type: **RTL Project**. Do not specify sources yet.
3. Target device: **xc7a100tcsg324-1**.
4. Finish the wizard.

[Figure placeholder]

`LH1E_part_selection.png`

Vivado "Default Part" dialog with `xc7a100tcsg324-1` highlighted.

Figure 2: Vivado "Default Part" dialog with `xc7a100tcsg324-1` highlighted.

4.4 Step 4 — Add the VHDL top module (switch-to-LED loopback)

1. In the *Sources* panel, click + → *Add or create design sources* → **Create File**.

   - *File type*: **VHDL**
   - *File name*: `loopback_top`
   - *File location*: <**Local to Project**>

   Click *OK*, then *Finish*. Dismiss the "Define Module" dialog with *OK*.

2. Double-click `loopback_top.vhd` and replace its entire contents with:

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity loopback_top is
    Port (
        sw  : in  STD_LOGIC_VECTOR(3 downto 0);
        led : out STD_LOGIC_VECTOR(3 downto 0)
    );
end loopback_top;

architecture Behavioral of loopback_top is
begin
    led <= sw;
end Behavioral;
```

Listing 1: `loopback_top.vhd` — switch-to-LED direct assignment.

Save (`Ctrl+S`). The entity `loopback_top` should appear as the top-level design unit in the *Sources* panel with no error markers.

4.5 Step 5 — Add and configure the constraint file

1. In the *Sources* panel, click + → *Add or create constraints* → *Add Files*.
2. Navigate to `Nexys-A7-100T-Master.xdc`, ensure **Copy sources into project** is checked, and click *Finish*.
3. Double-click the copied `.xdc` file and uncomment *exactly* the lines below (remove the leading `#` on each). All other lines must remain commented out.

```
## Slide Switches
set_property -dict { PACKAGE_PIN J15  IOSTANDARD LVCMOS33 } [get_ports { sw[0] }];
set_property -dict { PACKAGE_PIN L16  IOSTANDARD LVCMOS33 } [get_ports { sw[1] }];
set_property -dict { PACKAGE_PIN M13  IOSTANDARD LVCMOS33 } [get_ports { sw[2] }];
set_property -dict { PACKAGE_PIN R15  IOSTANDARD LVCMOS33 } [get_ports { sw[3] }];

## LEDs
set_property -dict { PACKAGE_PIN H17  IOSTANDARD LVCMOS33 } [get_ports { led[0] }];
set_property -dict { PACKAGE_PIN K15  IOSTANDARD LVCMOS33 } [get_ports { led[1] }];
set_property -dict { PACKAGE_PIN J13  IOSTANDARD LVCMOS33 } [get_ports { led[2] }];
set_property -dict { PACKAGE_PIN N14  IOSTANDARD LVCMOS33 } [get_ports { led[3] }];
```

Listing 2: Active constraint lines for the switch-to-LED loopback. Copy these exact `set_property` statements into your `.xdc`.

Save (`Ctrl+S`).

Do not uncomment extra pins

Leaving additional pins active causes port-mismatch errors during synthesis because the top-level entity does not declare those ports. The eight lines above are the *only* ones that should be uncommented.

4.6 Step 6 — Synthesize, implement, generate bitstream

1. Flow Navigator → **Run Synthesis**. Wait for completion with zero errors.
2. Flow Navigator → **Run Implementation**. Wait for completion.
3. Flow Navigator → **Generate Bitstream**. Output: `loopback_top.bit`.
4. Open the **Utilization** report. Expect single-digit LUT usage out of 63,400. If you see more than 10 LUTs, you likely have unconstrained ports driving logic.

4.7 Step 7 — Program and verify the switch-to-LED loopback

1. Connect the Nexys A7 via USB, power on. Open Hardware Manager → *Open Target* → *Auto Connect*.
2. Right-click the XC7A100T → *Program Device* → select `loopback_top.bit` → *Program*.
3. The DONE LED should illuminate.
4. Flip `SW0`: `LD0` should light. Flip `SW1`–`SW3` and verify `LD1`–`LD3` independently.
5. Power-cycle the board: the bitstream is volatile — all LEDs should go dark.

[Figure placeholder]

`LH1E_board_leds.jpg`

Nexys A7 with two slide switches up and the corresponding LEDs lit. DONE LED also illuminated.

Figure 3: Nexys A7 with two slide switches up and the corresponding LEDs lit. DONE LED also illuminated.

4.8 Step 8 — Add the UART echo loopback top module

The Nexys A7 connects its USB-UART bridge (Silicon Labs CP2102) to pins `E3` (TX to host) and `D3` (RX from host). Module 2 uses this path at 115,200 baud. This step verifies that path now so you are not debugging cabling during Module 2.

1. In the *Sources* panel, click + → *Add or create design sources* → **Create File**.
   - *File type*: **VHDL**
   - *File name*: `uart_echo_top`
2. Double-click `uart_echo_top.vhd` and replace its entire contents with:

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity uart_echo_top is
    Port (
        uart_rxd_out : out STD_LOGIC;    -- FPGA TX to USB-UART bridge (E3)
        uart_txd_in  : in  STD_LOGIC     -- FPGA RX from USB-UART bridge (D3)
    );
end uart_echo_top;

architecture Behavioral of uart_echo_top is
begin
    -- Direct loopback: echo every bit back immediately.
    -- No baud-rate logic needed; the bridge handles framing.
    uart_rxd_out <= uart_txd_in;
end Behavioral;
```

Listing 3: `uart_echo_top.vhd` — echoes every byte received from the host back to the host. No clock domain needed; this is a direct wire.

Save (`Ctrl+S`).

Why a direct wire works here

The CP2102 USB-UART bridge handles all framing and baud-rate generation. From the FPGA's perspective the signal is already a single-ended logic line that can be looped back without any internal logic. A proper UART receiver (`uart_rx.vhd`) is built in LH2-B; this step only confirms the physical path is wired correctly.

4.9 Step 9 — Constrain, synthesize, and program the UART echo design

1. Open the `.xdc` file. Comment out the eight switch/LED lines from Step 5 and add the following two lines instead:

```
## USB-UART (Silicon Labs CP2102 on Nexys A7)
set_property -dict { PACKAGE_PIN D4   IOSTANDARD LVCMOS33 } [get_ports {
    uart_txd_in  }];
set_property -dict { PACKAGE_PIN C4   IOSTANDARD LVCMOS33 } [get_ports {
    uart_rxd_out }];
```

Listing 4: UART pin constraints to add for the echo loopback. Comment out the sw/led lines first.

2. In the *Sources* panel, right-click `uart_echo_top` and select **Set as Top**.
3. Run Synthesis → Implementation → Generate Bitstream. Output: `uart_echo_top.bit`.
4. Program the Nexys A7 with `uart_echo_top.bit`.
5. Open your serial terminal at **115200 baud, 8N1, no flow control**, connected to the Nexys A7's COM port.
6. Type any characters. Each character you type should appear immediately in the terminal (echo). If nothing appears, check:
   - The correct COM port is selected (Device Manager → Ports → Silicon Labs CP210x).
   - The board is programmed (DONE LED on).
   - Local echo in the terminal is *off* so you are seeing FPGA-echoed characters, not the terminal's own echo.

Pin numbers vary by board revision

The Nexys A7-100T uses `D4`/`C4` for the CP2102 UART. If you have a Nexys A7-50T, the pins may differ. Always verify against the master XDC for your exact board SKU — the `uart_rxd_out` and `uart_txd_in` net names in the master file are the authoritative source.

5 Verification Checklist

- ☐ Vivado project created targeting `xc7a100tcsg324-1`.
- ☐ Switch-to-LED loopback: synthesis, implementation, and bitstream with zero errors.
- ☐ Switch-to-LED loopback: all four switches independently control their corresponding LEDs on hardware.
- ☐ Power cycle clears the bitstream (expected for volatile configuration).
- ☐ UART echo loopback: bitstream generated and programmed.
- ☐ UART echo loopback: characters typed in the terminal at 115200 baud return immediately.

Common Pitfalls

- **Constraint not found errors:** you edited the wrong `.xdc` file or added it as a design source rather than a constraints source.
- **Hardware Manager does not see the board:** on Linux, cable drivers must be installed manually (see `<Vivado>/data/xicom/cable_drivers/`). On Windows, re-run the installer and select *Install Cable Drivers*.
- **Device programs but LEDs do not respond:** the power switch is off, or the `JP3` jumper is set to wall-power but no wall adapter is connected.
- **UART echo: terminal shows nothing:** local echo is on (you are seeing your own keystrokes), or the wrong COM port is selected. Try toggling local echo off and retyping.
- **UART echo: random garbage characters:** baud rate mismatch. Confirm the terminal is set to exactly 115200.

Lab Handout 1-E Deliverable

1. `loopback_top.vhd` and its pruned `.xdc` (8 active lines).
2. `uart_echo_top.vhd` and its pruned `.xdc` (2 active lines).
3. Screenshot of the Vivado utilization report for either design (single-digit LUT usage expected).
4. Photo or short video showing each switch controlling its LED.
5. Screenshot of the serial terminal showing typed characters echoed back by the FPGA at 115200 baud.

Glossary

FPGA	Field-Programmable Gate Array	1, 7, 8, 9
RF	Radio Frequency	1
RTL	Register-Transfer Level	1
VHDL	VHSIC Hardware Description Language	1, 4